

BonusHashGridDemo

The Sample Project Mini-Cookbook

This FREE booklet is designed to accompany the sample project `BonusHashGridDemo` from the book [Practical C++ Game Programming with Data Structures and Algorithms](#). It provides additional insights into the code logic, algorithms, and problem-solving approaches used in the implementation of this sample.

In this mini booklet, you'll find how the sample project works and the key ideas behind its design.

Before we begin

You will need the knowledge of [Chapter 1](#), [Chapter 2](#) and [Chapter 4](#), or basic understanding of C++/raylib, and common data structure like array/list.

Getting project up and running

First, make sure you have pulled the latest updates from GitHub (<https://github.com/PacktPublishing/Practical-C-Game-Programming-with-Data-Structures-and-Algorithms/tree/main>).

Open the Visual Studio solution file in Visual Studio Community 2022, select the `BonusHashGridDemo` project as start-up project, rebuild, and run. You should see the similar scene like [Figure 1](#). You can use arrow keys to move the camera, rotate camera with mouse moving while holding right button down, or using mouse wheel to zoom.

Try to use mouse left button to click on any of the cubes. The cube will turn green and the surrounding cubes within predefined radius will turn orange. The range is visualized as yellow wired sphere.

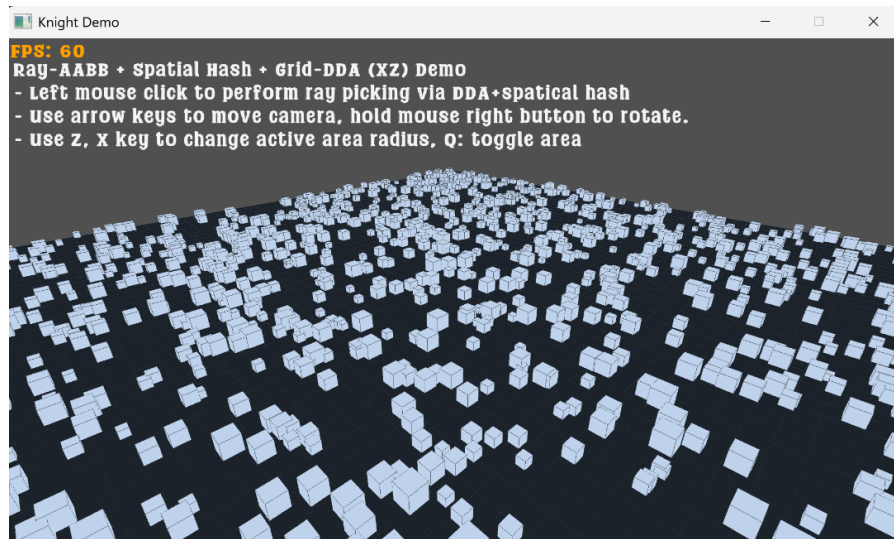


Figure 1 – Running the sample project BonusKdTreeDemo

1500 cubes are randomly moving around. Click any of them and it will start to query surrounding cubes within a certain radius. The radius is visualized as a yellow wired sphere like **Figure 2**.

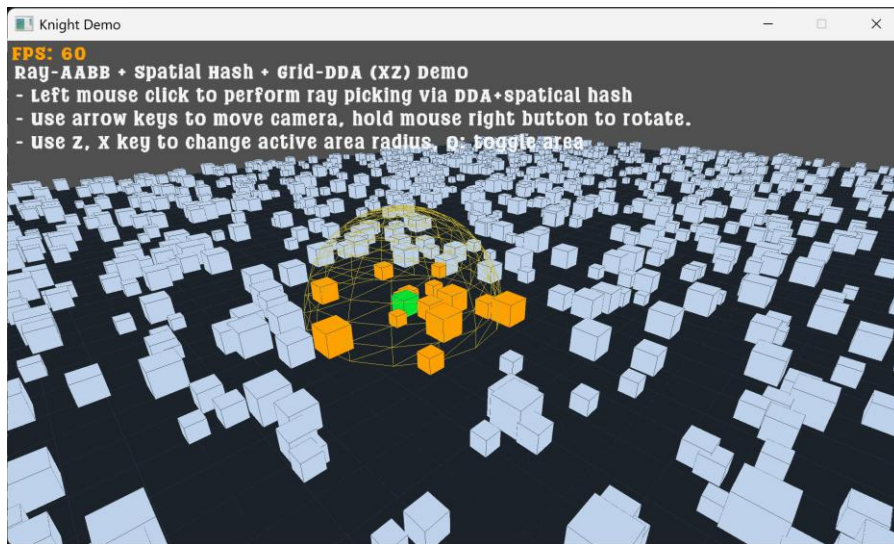


Figure 2 – Performing area query of surrounding cubes in realtime

You can use keyboard key **X/Z** to enlarge or shrink the radius (like in **Figure 3**). Click on any other cube will switch the area query focus on the newly selected cube.

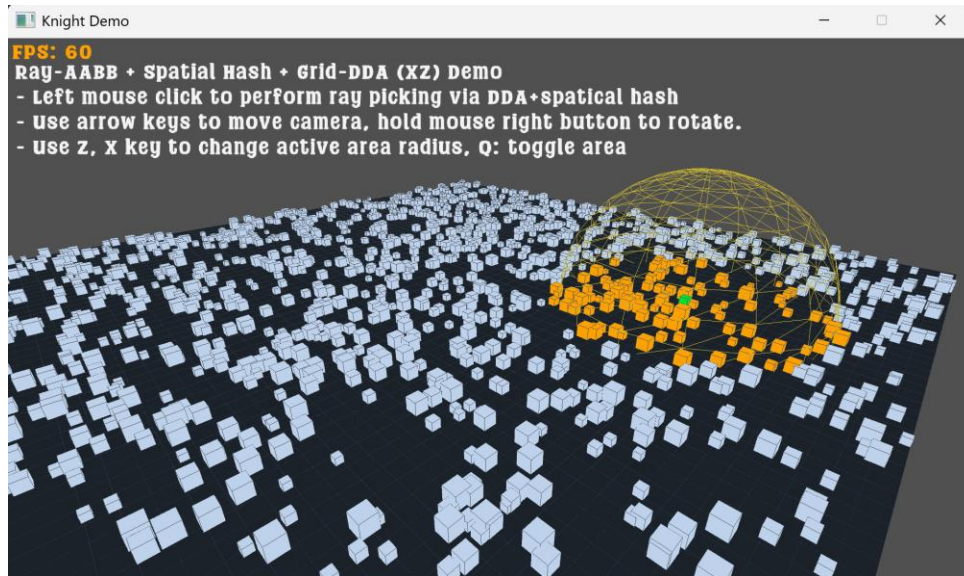


Figure 3 – Scaling the range of area to search surrounding cubes

Spatial hash and its use in game programming

A **hash** function is any function that can be used to map data of arbitrary size to fixed-size values. We have seen its simple use as **cache** to identify texture instance from its loaded path name in **Chapter 4**.

A **spatial hash** (or **spatial hashing**) is a data structure and technique for fast spatial queries — such as “find all objects near point P,” “what objects are under this ray,” or “which entities overlap this region.”

It’s especially useful in games and simulations where you have hundreds or thousands of moving objects in continuous 2D or 3D space. Such common usages include:

- Broad phase of collision detection
- Picking / ray tests

- Range queries (e.g. “nearby enemies”)
- Fluid/particle simulation
- AI “who’s nearby?” search
- Level-of-detail or culling

The core idea is instead of checking every object against every other object ($O(n^2)$ brute force), we divide the world into a uniform grid of cells — and we assign each object to the cell(s) it currently occupies.

But we don’t store the grid as a big array (most of it is empty). We hash the integer cell coordinates (like (x, z) or (x, y, z)) to an index in a hash table (e.g. `unordered_map` in C++ standard library).

So, it’s a sparse grid backed by a hash table — hence the name spatial hash.

How it works

We build the spatial hash data structure by the following 3 steps:

1. Quantize positions

Each object has a world position `pos`, and maybe a bounding sphere with radius or an axis aligned bounding box (AABB).

We then map the object’s world position to a “cell” coordinate by dividing by the grid’s cell size and flooring:

```
int cx = floor(pos.x / cellSize);  
int cz = floor(pos.z / cellSize);
```

2. Hash the cell coordinate

To store in a hash table, we mix these integers into a single key, for example:

```
size_t hash = (size_t)cx * 73856093u ^ (size_t)cz * 19349663u;
```

or we can do this in C++ as a custom `std::hash<Cell>` functor.

That hash key becomes the lookup index for the cell’s “bucket.”

3. Insert objects into the grid

Let's define a hash map in standard C++'s `std::unordered_map`:

```
std::unordered_map<Cell, std::vector<int>, CellHash> grid;
```

In each frame, or whenever an object moves:

- Clear the grid.
- Compute each object's cell.
- Push the object index into that cell's vector.

4. Query nearby objects efficiently

Now, when you want to find all objects within a region (say a circle or AABB):

- Compute which cells overlap that region.
- Only check objects in those few cells.

Likewise, for ray casting or collision detection, you can traverse only the cells the ray passes through.

In `BonusHashGridDemo`, we demonstrate two common challenges in real-time strategy games or large-scale MMO battle scenes—handling AI for hundreds of units efficiently using a spatial hash grid.

This Raylib/C++ sample spawns 1,500 moving cubes and showcases how a sparse spatial hash (grid) accelerates two key types of spatial queries:

- Ray picking (left mouse click): using a `grid-DDA` algorithm to walk cells along the ray and does precise Ray vs AABB tests only on cubes in visited cells.
- Area query (toggle key `Q`, adjust `Z/X`): highlight all cubes whose AABBs intersect a circle on XZ around a center point.

Walking through the demo

It's time to get some hands-on coding. Just like all other demo we introduced, all the magic happened in override functions of `Start()` and `Update()/DrawFrame()`.

Initializing the demo program

In the override `Start()` function:

- 1,500 `CubeEnt` are created with random position (XZ), velocity (XZ), and size (half). Y is set so each cube "sits on the ground." Every cube stores a world-space AABB via `min()/max()`.
- We define a `std::unordered_map<Cell, vector<int>, CellHash>` where `Cell = (x,z)` encodes grid cells on XZ. It's encoded with a strong 64-bit mix function hashes the pair.
- The demo use Knight's built in `FlyOverCamera` offers slightest better camera control.

Processing input and spatial queries

There are multiple steps happened in the per-frame `Update()` function:

The first part is process the keyboard input with key `X` increases area radius, and `Z` to decreases the radius. We clamped the range to [2, 40]. Key `Q` toggles area-query on/off.

We need to calculate the new position of all cubes in this frame. If it hit the boundary, it bounces back.

Now we perform the core "spatial hash" rebuilding step. We compute `cell = floor(p.x/cell), floor(p.z/cell)` for each cube with world space position `p` and push the cube index into that cell's bucket.

Here is also a hidden bonus feature. If you click middle mouse button within the ground boundary, it casts a ray from the camera through the cursor position and intersect it with the ground plane (`y=0`) to get a 3D point. Use that projected point as `areaCenter` and automatically turn area querying on. Instead of tracking cubes near a

selected moving cube, this allows us to track cubes move in/out the radius of a particular ground position.

Using 2D DDA to perform ray casting

Here we want to introduce another convenient algorithm when traverse with grid, or cell-based data structure.

DDA stands for **Digital Differential Analyzer**. It's a numerical algorithm used to step through discrete grid coordinates along a continuous line (or ray) in the correct order, without skipping any cells.

DDA Steps

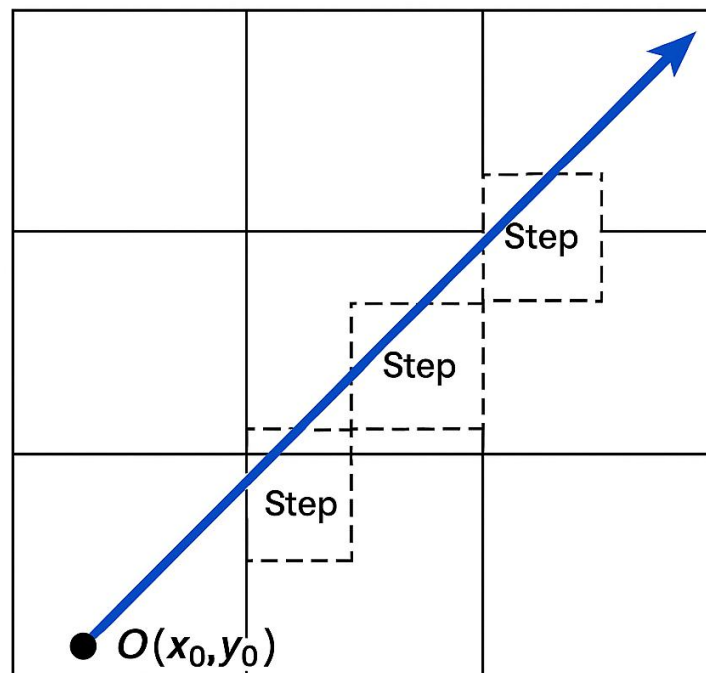


Figure 4 – How DDA steps through discrete grid coordinates

In simple terms: this algorithm converts a continuous geometric line (like $y = mx + b$ or a ray) into discrete integer steps (like pixels, cells, grids, tiles, or voxels) that approximate it. For people with computer graphics backgrounds, it's the core algorithm you learn how to draw a line in a pixel raster map. Famous games like *Wolfenstein 3D*, *Doom* also use it for extensive ray casting.

Let's say you have a grid made of square cells and a ray starting from point $O(x_0, y_0)$ with direction $D(dx, dy)$. We want to find which cells the ray passes through, and in what order — until it hits something or exits the grid. Instead of moving the ray in tiny floating-point steps, DDA jumps from one grid boundary to the next, analytically, so it's exact and fast.

Here in the code, we build a ray from the camera through the mouse. First, we need to grouping some variables to work with the algorithm:

```
// DDA on XZ plane
struct DDA2D {
    int ix, iz, stepX, stepZ; //initial (ix,iz), step (stepX, stepZ)
    float t, tMaxX, tMaxZ, tDeltaX, tDeltaZ;
};
```

We start a 2D DDA on XZ (inline function `InitDDA_XZ()`) in `BonusHashGrdDemo.h`:

```
static inline DDA2D InitDDA_XZ(const Ray& ray, float tStart, float
cellSize)
{
    DDA2D d;
    d.t = tStart;
    Vector3 p = Vector3Add(ray.position, Vector3Scale(ray.direction,
tStart));
    d.ix = (int)std::floor(p.x / cellSize);
    d.iz = (int)std::floor(p.z / cellSize);
    d.stepX = (ray.direction.x > 0.0f) ? 1 : -1;
    d.stepZ = (ray.direction.z > 0.0f) ? 1 : -1;
```



```

if (std::fabs(ray.direction.x) < 1e-8f) {
    d.tMaxX = d.tDeltaX = 1e30f;
}
else {
    float edgeX = (d.stepX > 0) ? ((d.ix + 1) * cellSize) : (d.ix *
cellSize);
    d.tMaxX = tStart + (edgeX - p.x) / ray.direction.x;
    d.tDeltaX = cellSize / std::fabs(ray.direction.x);
}
if (std::fabs(ray.direction.z) < 1e-8f) {
    d.tMaxZ = d.tDeltaZ = 1e30f;
}
else {
    float edgeZ = (d.stepZ > 0) ? ((d.iz + 1) * cellSize) : (d.iz *
cellSize);
    d.tMaxZ = tStart + (edgeZ - p.z) / ray.direction.z;
    d.tDeltaZ = cellSize / std::fabs(ray.direction.z);
}
return d;
}

```

This following inline function `DDA_Step_XZ()` takes discrete steps along the direction:

```

static inline void DDA_Step_XZ(DDA2D& d)
{
    if (d.tMaxX < d.tMaxZ) {
        d.ix += d.stepX; d.t = d.tMaxX; d.tMaxX += d.tDeltaX;
    } else {
        d.iz += d.stepZ; d.t = d.tMaxZ; d.tMaxZ += d.tDeltaZ;
    }
}

```

```
}

```

Now back to override `Update()` function in class `BonusHashGridDemo`. The following code snippets perform the core magic:

```
if (IsMouseButtonPressed(MOUSE_LEFT_BUTTON)) {
    Ray ray = GetMouseRay(GetMousePosition(), *camera->GetCamera3D());
    float tStart = 0.0f;
    if (std::fabs(ray.direction.y) > 1e-6f) {
        float tPlane = -ray.position.y / ray.direction.y;
        if (tPlane > 0.0f) tStart = tPlane;
    }
}
```

Once player click mouse left button, the above code computes a `tStart`: if the ray crosses the ground plane in front of the camera, start from there.

```
const float tEnd = 250.0f;
DDA2D dda = InitDDA_XZ(ray, tStart, CELL);
float bestT = tEnd + 1.0f;
int bestIdx = -1;
for (int steps = 0; steps < 3000; ++steps) {
    if (dda.t > tEnd) break;
    // hash lookup for current cell
    std::unordered_map<Cell, std::vector<int>, CellHash>::const_iterator
    it = grid.find(Cell{ dda.ix, dda.iz });
    if (it != grid.end()) {
        const std::vector<int>& bucket = it->second;
        for (size_t k = 0; k < bucket.size(); ++k) {
            int idx = bucket[k];
            float tHit;
            if (RayAABB(ray, ents[idx].min(), ents[idx].max(), tHit)) {
                if (tHit < bestT) { bestT = tHit; bestIdx = idx; }
            }
        }
    }
}
```

```

    }
}
if (bestIdx >= 0 && bestT <= dda.t)
    break; //hit before next boundary
}
DDA_Step_XZ(dda);
} // end of for step < 3000 loop

```

The above code is long but logic is simple: try to figure out which cell the ray is in at `tStart`, steps in `X/Z`, `tMaxX/tMaxZ`, and `tDeltaX/tDeltaZ` times to next cell boundary and per-cell increments.

Traverse cells until `tEnd` (250 units) or until an earlier hit is guaranteed:

- For the current cell, look up the bucket in grid.
- For each cube index in the bucket, run Ray–AABB test; keep the closest `tHit`.
- If the best hit `t` is before the next cell boundary (`bestT <= dda.t`), you can stop early.
- Otherwise step the DDA to the next neighboring cell (advance the smaller of `tMaxX/tMaxZ`).

If a hit is found, store its index (`selectIdx`).

Performing area query to enumerate neighboring cubes

Now we have made the spatial hash grid ready for action! If any cube or ground point is selected, we turn area-query on and set `areaCenter` = `selected.pos`.

The following code snippets demonstrate how to query the nearby cubes. We perform the area query (circle vs AABBs) using the hash grid:

- Compute a box of candidate cells that the circle of radius `areaRadius` around `areaCenter` could overlap.
- For each such cell, check only cubes in that bucket; run circle-AABB intersection on XZ plane.

```
// Area query via hash (circle in XZ vs each cube AABB)
if (areaActive) {
    int minCx = (int)std::floor((areaCenter.x - areaRadius) / CELL);
    int maxCx = (int)std::floor((areaCenter.x + areaRadius) / CELL);
    int minCz = (int)std::floor((areaCenter.z - areaRadius) / CELL);
    int maxCz = (int)std::floor((areaCenter.z + areaRadius) / CELL);
    for (int cx = minCx; cx <= maxCx; ++cx) {
        for (int cz = minCz; cz <= maxCz; ++cz) {
            std::unordered_map<Cell, std::vector<int>,
CellHash>::const_iterator it = grid.find(Cell{ cx, cz });
            if (it == grid.end()) continue;
            const std::vector<int>& bucket = it->second;
            for (size_t k = 0; k < bucket.size(); ++k) {
                int idx = bucket[k];
                if (CircleIntersectsAABB_XZ(areaCenter, areaRadius,
ents[idx].min(), ents[idx].max()))
                    ents[idx].highlight = true;
            }
        }
    }
}
```

In the above code snippets, the function to check if an AABB is overlapped with a circle is defined in the inline function `CircleIntersectsAABB_XZ()` implementation:

```
static inline bool CircleIntersectsAABB_XZ(const Vector3& center, float
radius, const Vector3& bmin, const Vector3& bmax)
{
    // distance from circle center to AABB on XZ
```

```
float dx = std::max(0.0f, std::max(bmin.x - center.x, center.x -
bmax.x));

float dz = std::max(0.0f, std::max(bmin.z - center.z, center.z -
bmax.z));

return (dx * dx + dz * dz) <= radius * radius;
}
```

The result of area query is visualized in the `DrawFrame()` function. For each cube, we rendered it as GREEN color if it's the currently selected cube, or ORANGE color if the cube is near the selected cube or selected ground point.

Conclusion

The spatial hash grid offers an effective way to perform certain spatial queries. It's fast because:

- **Locality:** Both picking and area queries only touch cubes inside a few grid cells instead of all 1,500.
- **Early-exit DDA:** The ray march stops as soon as it can prove there's no nearer hit beyond the next cell boundary.
- **Tight tests:** Final intersection is accurate Ray-AABB (not just cell hit).

Challenging yourself

To demonstrate the basic idea of sparse spatial hash, we try to keep the demo simple. However, don't stop here – go with further optimization ideas.

Here are some directions you can make even more efficient:

- Using incremental updates, track each cube's previous cell and only update when it crosses a boundary.

FREE mini-cookbook for sample project from Practical C++ Games Data Structure and Algorithms

- Think about the cell size. It does matter! Consider to set $CELL \approx \text{mean AABB size} * 2$. If you choose the size too small, it results with more cells; If you set the size per Cell too big, you end up big buckets.
- Combine with camera visibility optimization. We don't need to care the color of some invisible cubes from current camera view frustum right?

Happy coding!